



BINV1053

3.2 - Filtrage

Jérôme Plumat

Haute École Léonard de Vinci

2026





1 Introduction

2 Rappels

3 Introduction au Filtrage

4 Conclusion



1. Introduction

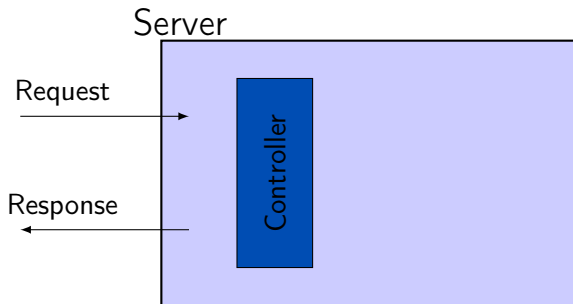


Objectifs

- Comprendre et implémenter la notion de filtrage.

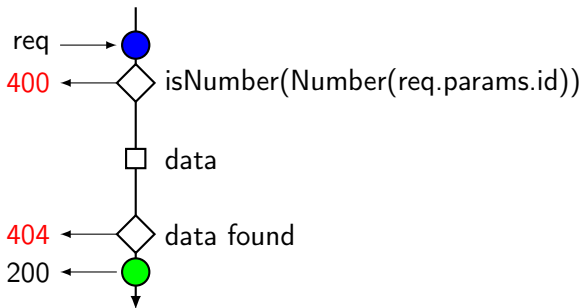


2. Rappels





Exemple : GET /doctors/1



La récupération des données (data) : **le service.**



3. Introduction au Filtrage



Certaines routes acceptent des paramètres pour filtrer les résultats. Par exemple :

- tous les docteurs d'une certaine spécialité ;
- tous les docteurs dont le nom commence par une certaine lettre ;
- etc.

Toutes ses requêtes peuvent être formulées en ajoutant des associations clé-valeur, formant la **query string** de l'URL.

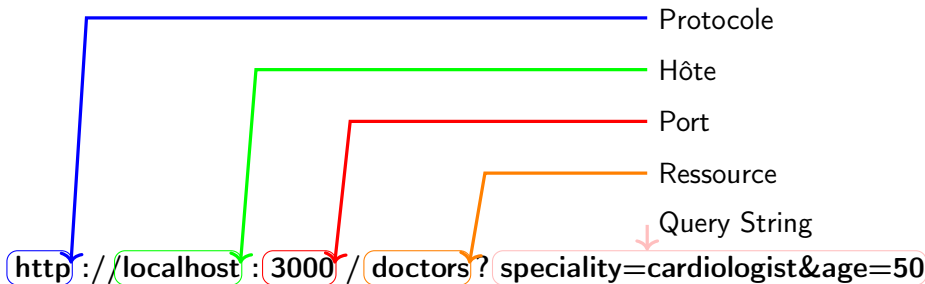


Figure – Description d'une URL avec une query.

La **query string** est la partie optionnelle de l'URL qui suit le `?`. Elle est composée de plusieurs associations clé-valeur séparées par des `&`. Dans l'exemple ci-dessus les clés sont `speciality` et `age`, et les valeurs sont `cardiologist` et `50` respectivement.



Exemples de filtrages appliqué à la liste des docteurs :

- `localhost:3000/doctors?speciality=cardiologist;`
 - ▶ tous les docteurs dont la spécialité est `cardiologist`;
- `localhost:3000/doctors?lastNamenname=said;`
 - ▶ tous les docteurs dont le nom de la famille est `said`;
- `localhost:3000/doctors?lastName=said&speciality=cardiologist;`
 - ▶ tous les docteurs dont le nom de la famille est `said` et dont la spécialité est `cardiologist`;
- etc.



Le filtrage consiste à **extraire les données de la query string** pour filtrer la liste des éléments renvoyés par le serveur.

Par exemple, on souhaite obtenir la liste des docteurs dont la spécialité est **cardiologist**, on va extraire la valeur de la clé **speciality** de la query string.



Le filtrage

Extraction des données (1)

Pour extraire les données de la query string, nous utiliserons le même principe que celui pour les paramètres de route. Nous utiliserons l'objet `req.query` qui contient les données de la query string.

Par exemple, pour récupérer la valeur de la clé `speciality` de l'URL `http://localhost:3000/doctors?speciality=cardiologist` :

```
1 router.get("/", (req: Request, res: Response) => {  
2   const specialityValue = req.query.speciality ; // extract potential value  
3   // of the key 'speciality'  
4 });
```



```
1 router.get("/", (req: Request, res: Response) => {  
2     const specialityValue = req.query.speciality ;  
3 });
```

Si nous avons appelé l'URL

`http://localhost:3000/doctors?speciality=cardiologist`, la variable `speciality` contiendra la valeur `cardiologist` et sera de type `string`.



Le filtrage

Extraction des données (3)

```
1 router.get("/", (req: Request, res: Response) => {  
2   const specialityValue = req.query.speciality ;  
3 });  
4
```

Si nous avons appelé l'URL `http://localhost:3000/doctors`, la variable `specialityValue` contiendra la valeur `undefined` car la clé `speciality` n'est pas présente dans la query string.

Toujours vérifier le type de la variable

La variable `specialityValue` peut être `undefined` si la clé `speciality` n'est pas présente dans la query string.

Il faut donc toujours vérifier la valeur de la variable avant de l'utiliser.



Le filtrage

Extraction des données (3)

Extraction de la valeur de la clé `speciality` et utilisation pour filtrer les éléments. On stocke la valeur dans un objet `filter` qui sera utilisé pour filtrer les éléments.

```
1 import { isString } from '../utils/guards';
2 import { DoctorFilter } from '../models/doctor.model';
3 router.get("/", (req: Request, res: Response) => {
4   const specialityValue = req.query.speciality;
5   const filter : DoctorFilter = {};
6   if ( isString ( specialityValue )) {
7     filter . speciality = specialityValue;
8   }
9   // ...
10 });
```

On prépare un objet `filter` de type `DoctorFilter` qui sera utilisée pour filtrer les éléments.



Dans le fichier `models/doctor.model.ts` :

```
1 export interface DoctorFilter {  
2     speciality?: string;  
3     // ...  
4 };
```

Notez le **?** après le nom de la propriété. Cela signifie que la propriété est optionnelle.



De la même façon que pour les paramètres de route, le contrôleur est **responsable de la gestion des données**. Il est donc responsable du contrôle et de la gestion des variables présentes dans la query et utilisées pour le filtrage.

On retrouve la même logique de vérification que celles utilisées pour les paramètres présents dans le **body** de la requête.



4. Conclusion



- Le filtrage s'effectue à l'aide de la query string de l'URL.
- La query string se trouve après un `?` dans l'URL.
- La query string est composée de plusieurs associations clé-valeur séparées par des `&`.
- le contrôleur contrôle les données transmises par le client.